

Write Spark programs in Scala to solve the following problems. Do not create any data structures, such as hash maps or tree. Use functional, rather than empirical, programming style. That is, do not use traditional **for**, **while**, or **do-while** statements. Using **foreach** to print the result is fine.

I used the following imports, testing the code on the csl machines with the following main function:

```
package example

import org.apache.spark.SparkContext._
import scala.io._
import org.apache.spark.{ SparkConf, SparkContext }
import org.apache.spark.rdd._
import org.apache.log4j.Logger
import org.apache.log4j.Level
import scala.collection._

object App {
  /* ... functions q1, q2, and q3 are here ... */

  def main(args: Array[String]) {
    // Don't log a bunch of the junk
    Logger.getLogger("org").setLevel(Level.OFF)
    Logger.getLogger("akka").setLevel(Level.OFF)

    // Spark setup
    val conf = new SparkConf().setAppName("App")
    val sc = new SparkContext(conf)

    q1(sc)
    q2(sc)
    q3(sc)
  }
}
```

Q1[10]. The program reads a file full of integers and computes the number of times each integer that is divisible by 3 occurs.

Example input file: 1 3 10 3

6 6 9 6

Example output: 3 appears 2 times, 6 appears 3 times, 9 appears 1 times

Do not worry about sorting the result.

```

def q1(sc: SparkContext) = {
  // Line: int int int ...
  val integersRdd = sc.textFile("input/asgn05/q1_integers/")
    .flatMap(l => l.split(" ")) // Space delimited
    .map(_.toInt)                // Convert each to an int
    .filter(_%3 == 0)           // Only divisible by 3 allowed
    .countByValue().foreach({   // Print every value and its frequency
      case (int, ct) => println(s"$int appears $ct times")
      case _ => throw new IllegalArgumentException("You are the
problem... You should never be here!")
    })
}

```

Q2[10]. The program reads a file with employees and a file with departments. The program should print the employee name and department name for each employee. Use a **cartesian** product and **filter** to solve the problem. Note that your filter can have the syntax: `.filter{ case (emp, dep) => ... }`. This means that if the input is a tuple of two values, you can process each value individually. Similarly, you can use `.map{ case (emp, dep) => ... }` to format the output.

Example Employee file: John, 23 //John works in department 23

Bob, 55
Steward, 20
Elizabeth, 44

Example Department file: 23, Computer Science

20, Mechanical Engineering
44, Industrial Engineering
55, Electrical Engineering

Example Output: John, Computer Science

Steward, Mechanical Engineering
Bob, Electrical Engineering
Elizabeth, Industrial Engineering

```

def q2(sc: SparkContext) = {
  // Line: employee name, department id
  val employeeRdd = sc.textFile("input/asgn05/q2_employees/")
    .map(l => l.split(",")) // one array per line Array[String, String]
    .map({                  // trim the result and turn into a tuple

```

```

        case Array(s1, s2) => (s1.trim, s2.trim)
        case _ => throw new IllegalArgumentException("You are the
problem... You should never be here!")
    })

    // Line: department id, department name
    val departmentRdd = sc.textFile("input/asgn05/q2_departments/")
    .map(l => l.split(",")) // one array per line Array[String, String]
    .map({ // trim the result and convert into a tuple
        case Array(s1, s2) => (s1.trim, s2.trim)
        case _ => throw new IllegalArgumentException("You are the
problem... You should never be here!")
    })

    // Cartesian product of the two inputs
    // result in the format ((ename, did), (did, dname))
    var rdd = employeeRdd.cartesian(departmentRdd)
    .filter({ // join on department id
        // ((ename, did), (did2, dname))
        // empl._2 and dept._1 is the did
        case (empl, dept) => empl._2 == dept._1
        case _ => throw new IllegalArgumentException("You are the
problem... You should never be here!")
    })
    .collect().foreach({
        // ((ename, did), (did2, dname))
        // empl._1 is the employee name
        // dept._2 is the department name
        case (empl, dept) =>
            val ename = empl._1
            val dname = dept._2
            println(s"$ename, $dname")
        case _ => throw new IllegalArgumentException("You are the
problem... You should never be here!")
    })
}

```

Q3[10]. Consider the following input file:

```
Bob Wilson, 11, B CS201
John Back, 23, A CSC369, B CSC366
```

It contains the student name, student ID, and a list of courses the student has taken (grade and course). Write a program with the following example output.

```
Bob Wilson, 11, 3.0
John Back, 23, 3.5
```

The program prints the student name, student ID, and their GPA. You can assume that there is a single entry per student in the input file. Note that you can use `line.split(", ", 3)(2).trim()` to only consider the first two commas of a string and get everything after the second comma. Feel free to create a separate method that computes the GPA and a map that converts a letter grade to a numeric value.

Use the **aggregate** operator to solve the problem.

```
def q3(sc: SparkContext) = {
  // Raw input lines from a text file
  val studentsRdd = sc.textFile("input/asgn05/q3_students/")
  .map(l => l.split(", ", 3))    // Parse to (name, id, gradeCourses)
  .map({                          // Convert into a tuple
    case Array(name, id, gradeCourse) => (name, id, gradeCourse)
    case _ => throw new IllegalArgumentException("You are the
problem... You should never be here!")
  })
  .map({                          // Extract the letter grades and get
gpa
    // Split every grade based on the comma
    // Extract the grade letter for every course (the first char of the
grade course pair)
    // Calculate the gpa based on those letters
    // result in the format of (name, id, grades)
    case (name, id, gradeCourses) =>
      val letters = gradeCourses
        .split(",")    // Split into an Array of Strings " A CSC400"
        .map(_.trim)   // For each of them, trim it down "A CSC400"
        .map(_.substring(0, 1)) // Get the first character of each "A"
      (name, id, getGpa(letters)) // Return a tuple of the GPA
    case _ => throw new IllegalArgumentException("You are the
problem... You should never be here!")
  })
}
```

```

        .collect().foreach({          // Print the result
            case (name, id, gpa) =>
                println(s"$name, $id, $gpa")
        })
    }

    // Returns the average GPA from a list of letter grades
    def getGpa(gs: Array[String]): Double = {
        // Maps all the letters to a numerical value, then sums, then divides
        // by the
        // number of strings that were given

        val result = gs
            .map(x => lToNGrade(x))
            .aggregate((0.0, 0.0))(
                (x,y) => (x._1 + y, x._2 + 1),          // At each node, get sum and
count
                (x,y) => (x._1 + y._1, x._2 + y._2))    // Collectively combine date
from every node

        return result._1/result._2
    }

    def lToNGrade(l: String): Double = {
        return l match {
            case "A" => 4
            case "a" => 4
            case "B" => 3
            case "b" => 3
            case "C" => 2
            case "c" => 2
            case "D" => 1
            case "d" => 1
            case "F" => 0
            case "f" => 0
            case _ => Double.NaN
        }
    }
}

```